Babbage's Hall

A Victorian hall of machines that compute · 19 October 2026

A Victorian engineering hall of machines that compute: cast-iron columns painted Brunswick green, with gilded capitals, gas lamps, a glass barrel vault, red brick, brass rails and encaustic tiles. **Every machine in it can be 3D printed and built**: each exhibit's folder holds its OpenSCAD sources, a script that makes its STL files, and a README saying what else to buy and how to put it together. What is printed is what is simulated.

Down the west side are machines that **count**, down the east side machines that **measure**. The bays still waiting for their machines stand covered, "in preparation". When an exhibit's script turns up in its folder, its bay comes to life.

Bay	Exhibit	
west 1	**The Domino Half Adder**: dominoes that add two bits	`exhibits/DominoAdder/`
east 1	**The Differential**: bevel gears that add and subtract	`exhibits/Differential/`
west 2	The Digi-Comp II	in preparation
east 2	Kelvin's tide predictor	in preparation
west 3	**The Pascaline**: Pascal's adding machine, with his falling-weight carry	`exhibits/Pascaline/`
east 3	The Differential Analyser	in preparation
west 4	The Difference Engine	in preparation
east 4	Lehmer's number sieve	in preparation

Running it

Open `hall.lua` in bpp, or from the repository's top folder run `release/bpp -f demo/WyomingWill/BabbagesHall/hall.lua`. Each exhibit also runs on its own from its folder.

Key	
`Tab`	walk on: the view from the entrance, the exhibits, the view from the title wall
`Shift+Tab`	walk back

Click the 3D view first so it has the keyboard. Everything else goes to the exhibit you're at, with the same keys as when it runs on its own; the Shortcuts pane lists them. `HALL_START=differential` (any part of a name) starts at that exhibit, and `HALL_START=end` at the title wall.

How it works

The machinery is Euler's Gallery's, unchanged; the building around it is new.

- **Separate worlds.** Each exhibit runs in a Bullet world of its own, at its own coordinates, drawn in its bay. Only the exhibit you are standing at is stepped.
- **Sandboxes.** Each exhibit is loaded with its own globals, its own random numbers, and its own sliders, keys and help.

- **Files.** An exhibit finds its files from its own folder. `hallPath(name)` gives a file's path from bpp's folder; the exhibits use it to hand OpenSCAD the full path to their parts files.
- **Requirement:** a bpp with separate worlds (`v:newWorld`).

The placards' lettering is made by `python3 make-labels.py` (needs Pillow, NumPy and the DejaVu fonts) into `hall-meshes/`.

Every `.md` file has a typeset `.pdf` beside it; `./make-pdfs.sh` rebuilds them all (needs `pip install markdown playwright`). The STL files are not kept in the repository: each exhibit's `make-stls.sh` makes them from its OpenSCAD sources.

Notes for exhibit authors

Two things about bpp that cost time, and how the exhibits handle them:

- **OpenSCAD caching.** bpp caches each OpenSCAD part by the text asked for, not by what the included files say, so an edited `.scad` file is not seen. The exhibits put a checksum of their parts files into that text.
- **Timing of settings.** An OpenSCAD mesh's place has to be set after it is added to the world. Its `collides` has to be set before: collision groups are fixed when a body is added.
- **Gear contact.** Bevel teeth as meshes (GImpact) jammed and slipped. As compounds of convex hulls (one per tooth, plus the body) they mesh well. See `exhibits/Differential/FINDINGS.md`; `parts/make-hulls.py` there makes them from the same OpenSCAD source.
- **`.scad` files and git.** bpp's top-level `.gitignore` ignores `*.scad`, so a new exhibit's parts files are silently left out of a commit. The hall's own `.gitignore` has `!*.scad` to keep them; check with `git status` that they are there.
- **One source of geometry.** The Pascaline keeps every moving part's shape as primitives in a Lua file; the simulation builds its bodies from them and writes `geometry.scad`, from which OpenSCAD builds the printed parts.
- **Centre of mass.** A Bullet body's origin is where gravity acts, so a lever built from an off-centre mesh must have its origin moved to its centre of mass (the Pascaline's `bodyOf` does this).
- **Hinge angles.** `getHingeAngle` is clockwise-positive about the hinge's axis, the opposite of a right-handed turn.